

Secure EC2 Web Server

A hands-on AWS project: hosting a portfolio site on Nginx inside a custom VPC.

By Jervil Angelo D. De Leon

Overview

While studying for the AWS Certified Cloud Practitioner (CLF-C02) exam, I wanted to practice the services I was reading about instead of only memorizing them. I built a small but secure web server on Amazon EC2, placed it in a network I designed myself, and used it to host my portfolio website.

Goals

- Apply EC2, VPC, security groups, IAM, and EBS in one working project.
- Follow security basics: no daily work as root, restricted access, and cost controls.
- Document the work so others can follow it.

Architecture

A visitor's browser reaches the internet gateway, which routes traffic to a public subnet inside the VPC. The EC2 instance in that subnet runs Nginx and is protected by a security group.

Region	Asia Pacific (Singapore), ap-southeast-1
VPC	Custom VPC, 10.0.0.0/16, one Availability Zone
Subnet	One public subnet with auto-assign public IPv4 enabled
Routing	Internet gateway attached; public route table sends 0.0.0.0/0 to the gateway
Compute	EC2 instance on Amazon Linux 2023 running Nginx, installed with a user data script
Storage	EBS volume with a snapshot taken as a backup
Access	Security group plus an IAM role for Session Manager

What I did

Phase 1: Account setup	Picked one region, enabled MFA on the root account, created a monthly budget with an 80% email alert, turned on Free Tier alerts, and created an IAM user in an admin group with MFA for daily work.
Phase 2: Network	Created the VPC with one public subnet and no NAT gateway or VPC endpoints to avoid extra cost. Verified the internet gateway, created a public route table with a default route to it, and associated it with the subnet.
Phase 3: Server	Launched the instance, confirmed it passed its status checks, and verified Nginx from the browser. Connected over SSH with a key pair, uploaded the site with scp, moved it into the Nginx web folder, and set file permissions to 644.
Phase 4: Backup	Created an EBS snapshot of the instance volume and confirmed it reached the Completed state.

Security measures

- Root account protected with MFA and not used for daily work.
- Security group allows HTTP (port 80) from anywhere and SSH (port 22) from my IP address only.
- IAM role attached to the instance so Session Manager can be used as a keyless backup way in.
- No NAT gateway and a \$5 budget alert as cost guardrails.
- Private key kept off GitHub and never shared.

Problems and how I solved them

SSH: Permission denied (publickey)	The error log showed the key file was not found. I was running the command from the wrong folder. Using the full path to the key file, with quotes because of spaces in the folder name, fixed it.
Key file permissions on Windows	Windows requires the private key to be readable only by my user. I removed inherited permissions and granted read access to my account with icacfs.
sudo error in PowerShell	I had run server commands on my own PC instead of the SSH session. Linux commands such as sudo mv must run on the server, while scp runs on my computer.

Cloud concepts this reinforced

- Shared responsibility model: AWS secures the infrastructure, while I manage the OS, patches, and security group rules.
- Least privilege: open only the ports and permissions that are needed.
- Regions and Availability Zones: resources are tied to a region, so staying in one region avoids confusion.
- Cost awareness: public IPv4 addresses, NAT gateways, and snapshots can all create charges.

Next steps

- Add HTTPS with a domain name and certificate.
- Automate the deployment with a script or infrastructure as code.
- Add monitoring with CloudWatch.
- Rebuild the setup with CloudFormation or Terraform.
- Cleanup: stop the instance when not in use, and delete the instance, snapshot, and VPC when finished.